串

子串与子序列

中文名称	常见英文名称	解释
子串	substring	连续的选择一段字符（可以全选、可以不选）组成的新字符串
子序列	subsequence	从左到右取出若干个字符（可以不取、可以全取、可以不连续）组成的新字符串

字符串模式匹配算法 KMP

> 应用：
>
> 1. 在字符串中查找子串；
> 2. 最小周期：字符串长度-整个字符串的 border ；
> 3. 最小循环节：区别于周期，当字符串长度 $n \bmod (n - nxt[n]) = 0$ 时，等于最小周期，否则为 n 。

以最坏 $\mathcal{O}(N + M)$ 的时间计算 t 在 s 中出现的全部位置。

```
auto kmp = [&](string s, string t) {
    int n = s.size(), m = t.size();
    vector<int> kmp(m + 1), ans;
    s = "@" + s;
    t = "@" + t;
    for (int i = 2, j = 0; i <= m; i++) {
        while (j && t[i] != t[j + 1]) {
            j = kmp[j];
        }
        j += t[i] == t[j + 1];
        kmp[i] = j;
    }
    for (int i = 1, j = 0; i <= n; i++) {
        while (j && s[i] != t[j + 1]) {
            j = kmp[j];
        }
        if (s[i] == t[j + 1] && ++j == m) {
            ans.push_back(i - m + 1); // t 在 s 中出现的位置
        }
    }
    return ans;
};
```

Z函数（扩展 KMP）

获取字符串 s 和 $s[i,n-1]$（即以 $s[i]$ 开头的后缀）的最长公共前缀（LCP）的长度，总复杂度 $\mathcal{O}(N)$。

```
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n);
    z[0] = n;
    for (int i = 1, j = 1; i < n; i++) {
        z[i] = max(0, min(j + z[j] - i, z[i - j]));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > j + z[j]) {
            j = i;
        }
    }
    return z;
}
```

最长公共子序列 LCS

求解两个串的最长公共子序列的长度。

小数据解

针对 10^3 以内的数据。

```
const int N = 1e3 + 10;
char a[N], b[N];
int n, m, f[N][N];
void solve(){
    cin >> n >> m >> a + 1 >> b + 1;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++){
            f[i][j] = max(f[i - 1][j], f[i][j - 1]);
            if (a[i] == b[j]) f[i][j] = max(f[i][j], f[i - 1][j - 1] + 1);
        }
    cout << f[n][m] << "\n";
}
int main(){
    solve();
    return 0;
}
```

大数据解

针对 10^5 以内的数据。

```
const int INF = 0x7fffffff;
int n, a[maxn], b[maxn], f[maxn], p[maxn];
int main(){
    cin >> n;
    for (int i = 1; i <= n; i++){
        scanf("%d", &a[i]);
        p[a[i]] = i;  //将第二个序列中的元素映射到第一个中
    }
    for (int i = 1; i <= n; i++){
        scanf("%d", &b[i]);
        f[i] = INF;
    }
    int len = 0;
    f[0] = 0;
    for (int i = 1; i <= n; i++){
        if (p[b[i]] > f[len]) f[++len] = p[b[i]];
        else {
            int l = 0, r = len;
            while (l < r){
                int mid = (l + r) >> 1;
                if (f[mid] > p[b[i]]) r = mid;
                else l = mid + 1;
            }
            f[l] = min(f[l], p[b[i]]);
        }
    }
    cout << len << "\n";
    return 0;
}
```

字符串哈希

双哈希封装

随机质数列表：1111111121、1211111123、1311111119。

```
const int N = 1 << 21;
static const int mod1 = 1E9 + 7, base1 = 127;
static const int mod2 = 1E9 + 9, base2 = 131;
using U = Zmod<mod1>;
using V = Zmod<mod2>;
vector<U> val1;
vector<V> val2;
void init(int n = N) {
    val1.resize(n + 1), val2.resize(n + 2);
    val1[0] = 1, val2[0] = 1;
    for (int i = 1; i <= n; i++) {
```

```
        val1[i] = val1[i - 1] * base1;
        val2[i] = val2[i - 1] * base2;
    }
}
struct String {
    vector<U> hash1;
    vector<V> hash2;
    string s;

    String(string s_) : s(s_), hash1{1}, hash2{1} {
        for (auto it : s) {
            hash1.push_back(hash1.back() * base1 + it);
            hash2.push_back(hash2.back() * base2 + it);
        }
    }
    pair<U, V> get() { // 输出整串的哈希值
        return {hash1.back(), hash2.back()};
    }
    pair<U, V> substring(int l, int r) { // 输出子串的哈希值
        if (l > r) swap(l, r);
        U ans1 = hash1[r + 1] - hash1[l] * val1[r - l + 1];
        V ans2 = hash2[r + 1] - hash2[l] * val2[r - l + 1];
        return {ans1, ans2};
    }
    pair<U, V> modify(int idx, char x) { // 修改 idx 位为 x
        int n = s.size() - 1;
        U ans1 = hash1.back() + val1[n - idx] * (x - s[idx]);
        V ans2 = hash2.back() + val2[n - idx] * (x - s[idx]);
        return {ans1, ans2};
    }
};
```

前后缀去重

`sample please ease` 去重后得到 `samplease`。

```
string compress(vector<string> in) { // 前后缀压缩
    vector<U> hash1{1};
    vector<V> hash2{1};
    string ans = "#";
    for (auto s : in) {
        s = "#" + s;
        int st = 0;
        U chk1 = 0;
        V chk2 = 0;
        for (int j = 1; j < s.size() && j < ans.size(); j++) {
            chk1 = chk1 * base1 + s[j];
            chk2 = chk2 * base2 + s[j];
            if ((hash1.back() == hash1[ans.size() - 1 - j] * val1[j] + chk1) &&
                (hash2.back() == hash2[ans.size() - 1 - j] * val2[j] + chk2)) {
                st = j;
```

```
            }
        }
        for (int j = st + 1; j < s.size(); j++) {
            ans += s[j];
            hash1.push_back(hash1.back() * base1 + s[j]);
            hash2.push_back(hash2.back() * base2 + s[j]);
        }
    }
    return ans.substr(1);
}
```

马拉车

$\mathcal{O}(N)$ 时间求出字符串的最长回文子串。

```
string s;
cin >> s;
int n = s.length();
string t = "-#";
for (int i = 0; i < n; i++) {
    t += s[i];
    t += '#';
}
int m = t.length();
t += '+';
int mid = 0, r = 0;
vector<int> p(m);
for (int i = 1; i < m; i++) {
    p[i] = i < r ? min(p[2 * mid - i], r - i) : 1;
    while (t[i - p[i]] == t[i + p[i]]) p[i]++;
    if (i + p[i] > r) {
        r = i + p[i];
        mid = i;
    }
}
```

字典树 trie

基础封装

```
struct Trie {
    int ch[N][63], cnt[N], idx = 0;
    map<char, int> mp;
    void init() {
        LL id = 0;
        for (char c = 'a'; c <= 'z'; c++) mp[c] = ++id;
        for (char c = 'A'; c <= 'Z'; c++) mp[c] = ++id;
        for (char c = '0'; c <= '9'; c++) mp[c] = ++id;
    }
    void insert(string s) {
```

```
        int u = 0;
        for (int i = 0; i < s.size(); i++) {
            int v = mp[s[i]];
            if (!ch[u][v]) ch[u][v] = ++idx;
            u = ch[u][v];
            cnt[u]++;
        }
    }
    LL query(string s) {
        int u = 0;
        for (int i = 0; i < s.size(); i++) {
            int v = mp[s[i]];
            if (!ch[u][v]) return 0;
            u = ch[u][v];
        }
        return cnt[u];
    }
    void Clear() {
        for (int i = 0; i <= idx; i++) {
            cnt[i] = 0;
            for (int j = 0; j <= 62; j++) {
                ch[i][j] = 0;
            }
        }
        idx = 0;
    }
} trie;
```

01 字典树

```
struct Trie {
    int n, idx;
    vector<vector<int>> ch;
    Trie(int n) {
        this->n = n;
        idx = 0;
        ch.resize(30 * (n + 1), vector<int>(2));
    }
    void insert(int x) {
        int u = 0;
        for (int i = 30; ~i; i--) {
            int &v = ch[u][x >> i & 1];
            if (!v) v = ++idx;
            u = v;
        }
    }
    int query(int x) {
        int u = 0, res = 0;
        for (int i = 30; ~i; i--) {
            int v = x >> i & 1;
            if (ch[u][!v]) {
```

```
                res += (1 << i);
                u = ch[u][!v];
            } else {
                u = ch[u][v];
            }
        }
        return res;
    }
};
```

后缀数组 SA

以 $\mathcal{O}(N)$ 的复杂度求解。

```
struct SuffixArray {
    int n;
    vector<int> sa, rk, lc;
    SuffixArray(const string &s) {
        n = s.length();
        sa.resize(n);
        lc.resize(n - 1);
        rk.resize(n);
        iota(sa.begin(), sa.end(), 0);
        sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
        rk[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        }
        int k = 1;
        vector<int> tmp, cnt(n);
        tmp.reserve(n);
        while (rk[sa[n - 1]] < n - 1) {
            tmp.clear();
            for (int i = 0; i < k; ++i) {
                tmp.push_back(n - k + i);
            }
            for (auto i : sa) {
                if (i >= k) {
                    tmp.push_back(i - k);
                }
            }
            fill(cnt.begin(), cnt.end(), 0);
            for (int i = 0; i < n; ++i) {
                ++cnt[rk[i]];
            }
            for (int i = 1; i < n; ++i) {
                cnt[i] += cnt[i - 1];
            }
            for (int i = n - 1; i >= 0; --i) {
                sa[--cnt[rk[tmp[i]]]] = tmp[i];
            }
```

```
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; ++i) {
                rk[sa[i]] = rk[sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i -
1] + k == n ||
                                                tmp[sa[i - 1] + k] < tmp[sa[i] + k]);
            }
            k *= 2;
        }
        for (int i = 0, j = 0; i < n; ++i) {
            if (rk[i] == 0) {
                j = 0;
                continue;
            }
            for (j -= j > 0;
                 i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] +
j];) {
                ++j;
            }
            lc[rk[i] - 1] = j;
        }
    }
};
```

AC 自动机

定义 $|s_i|$ 是模板串的长度，$|S|$ 是文本串的长度，$|\Sigma|$ 是字符集的大小（常数，一般为 26），时间复杂度为 $\mathcal{O}(\sum |s_i| + |S|)$。

```
// Trie+Kmp，多模式串匹配
struct ACAutomaton {
    static constexpr int N = 1e6 + 10;
    int ch[N][26], fail[N], cntNodes;
    int cnt[N];
    ACAutomaton() {
        cntNodes = 1;
    }
    void insert(string s) {
        int u = 1;
        for (auto c : s) {
            int &v = ch[u][c - 'a'];
            if (!v) v = ++cntNodes;
            u = v;
        }
        cnt[u]++;
    }
    void build() {
        fill(ch[0], ch[0] + 26, 1);
        queue<int> q;
        q.push(1);
        while (!q.empty()) {
```

```
            int u = q.front();
            q.pop();
            for (int i = 0; i < 26; i++) {
                int &v = ch[u][i];
                if (!v)
                    v = ch[fail[u]][i];
                else {
                    fail[v] = ch[fail[u]][i];
                    q.push(v);
                }
            }
        }
    }
    LL query(string t) {
        LL ans = 0;
        int u = 1;
        for (auto c : t) {
            u = ch[u][c - 'a'];
            for (int v = u; v && ~cnt[v]; v = fail[v]) {
                ans += cnt[v];
                cnt[v] = -1;
            }
        }
        return ans;
    }
};
```

```
const int MAXN = 200005; // 根据题目要求调整大小

struct ACAutomaton {
    // 节点池大小
    static constexpr int N = MAXN;

    // ch: Trie转移, fail: 失配指针
    // cnt: 记录节点被访问的次数 (对应 Java 的 times[])
    // pos: 记录第 i 个模式串结束的位置 (对应 Java 的 end[])
    int ch[N][26], fail[N], cnt[N];
    int pos[N];
    int cntNodes;

    // Fail 树的邻接表 (对应 Java 的 head/next/to)
    vector<int> adj[N];
    // 入度数组（如果使用拓扑排序写法需要，这里用DFS，暂不需要）

    ACAutomaton() {
        init();
    }

    // 初始化/重置函数
    void init() {
        // 根据题目数据量，这里可能需要 memset 或循环清空
```

```
    // 为了安全起见，通常清空 ch, fail, cnt, adj 等
    // 这里假设是全局或单次运行，简单重置 cntNodes

    // 如果是多组数据，这里需要彻底清空，例如：
    memset(ch, 0, sizeof(ch[0])*(cntNodes+1));
    memset(fail, 0, sizeof(fail[0])*(cntNodes+1));
    memset(cnt, 0, sizeof(cnt[0])*(cntNodes+1));
    for(int i=0; i<=cntNodes; i++) adj[i].clear();
    cntNodes = 1;
}

// 插入模式串，id 为模式串的编号 (1-based)
void insert(const string& s, int id) {
    int u = 1;
    for (char c : s) {
        int idx = c - 'a';
        if (!ch[u][idx]) ch[u][idx] = ++cntNodes;
        u = ch[u][idx];
    }
    pos[id] = u; // 记录第 id 个串结束在节点 u
}

// 构建 Fail 指针 (Trie 图)
void build() {
    // 初始化 0 号节点的子节点指向 1 (虚拟根节点的处理)
    for (int i = 0; i < 26; i++) ch[0][i] = 1;

    queue<int> q;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = 0; i < 26; i++) {
            int &v = ch[u][i];
            if (!v) {
                // 路径压缩/Trie图优化：如果不存在该子节点，直接指向 fail 的对应子节点
                v = ch[fail[u]][i];
            } else {
                fail[v] = ch[fail[u]][i];
                q.push(v);
            }
        }
    }
}

// 处理文本串，只打标记，不统计
void query(const string& t) {
    int u = 1;
    for (char c : t) {
        u = ch[u][c - 'a']; // 在 Trie 图上移动
```

```
            cnt[u]++;           // 当前节点被访问次数 +1
        }
    }

    // 核心逻辑：建立 Fail 树并汇总结果
    // 对应 Java 中的 addEdge 和 f2 逻辑
    void buildFailTreeAndCalc() {
        // 1. 建树：fail[v] -> v
        // 根节点是 1，fail[1] 是 0，通常我们只需要从 2 开始遍历
        for (int i = 2; i <= cntNodes; i++) {
            adj[fail[i]].push_back(i);
        }

        // 2. DFS 统计子树和
        dfs(1);
    }

    // 递归 DFS 统计词频
    // C++ 默认栈空间一般够用 (2e5 层深度通常没问题)
    // 如果担心爆栈，可以使用拓扑排序代替 DFS
    void dfs(int u) {
        for (int v : adj[u]) {
            dfs(v);
            cnt[u] += cnt[v]; // 子节点的出现次数累加给父节点 (Fail 指针指向的节点)
        }
    }

    // 获取第 id 个模式串的出现次数
    int getAnswer(int id) {
        return cnt[pos[id]];
    }
};
```

回文自动机 PAM（回文树）

> 应用：
>
> 1. 本质不同的回文串个数：$idx-2$；
> 2. 回文子串出现次数。

对于一个字符串 s，它的本质不同回文子串个数最多只有 $|s|$ 个，那么，构造 s 的回文树的时间复杂度是 $\mathcal{O}(|s|)$。

```
struct PalindromeAutomaton {
    constexpr static int N = 5e5 + 10;
    int tr[N][26], fail[N], len[N];
    int cntNodes, last;
    int cnt[N];
    string s;
    PalindromeAutomaton(string s) {
        memset(tr, 0, sizeof tr);
```

```
        memset(fail, 0, sizeof fail);
        len[0] = 0, fail[0] = 1;
        len[1] = -1, fail[1] = 0;
        cntNodes = 1;
        last = 0;
        this->s = s;
    }
    void insert(char c, int i) {
        int u = get_fail(last, i);
        if (!tr[u][c - 'a']) {
            int v = ++cntNodes;
            fail[v] = tr[get_fail(fail[u], i)][c - 'a'];
            tr[u][c - 'a'] = v;
            len[v] = len[u] + 2;
            cnt[v] = cnt[fail[v]] + 1;
        }
        last = tr[u][c - 'a'];
    }
    int get_fail(int u, int i) {
        while (i - len[u] - 1 <= -1 || s[i - len[u] - 1] != s[i]) {
            u = fail[u];
        }
        return u;
    }
};
```

后缀自动机 SAM

定义 $|\Sigma|$ 是字符集的大小，复杂度为 $\mathcal{O}(N \log |\Sigma|)$ 。

```
// 有向无环图
struct SuffixAutomaton {
    static constexpr int N = 1e6;
    struct node {
        int len, link, nxt[26];
        int siz;
    } t[N << 1];
    int cntNodes;
    SuffixAutomaton() {
        cntNodes = 1;
        fill(t[0].nxt, t[0].nxt + 26, 1);
        t[0].len = -1;
    }
    int extend(int p, int c) {
        if (t[p].nxt[c]) {
            int q = t[p].nxt[c];
            if (t[q].len == t[p].len + 1) {
                return q;
            }
            int r = ++cntNodes;
            t[r].siz = 0;
```

```
            t[r].len = t[p].len + 1;
            t[r].link = t[q].link;
            copy(t[q].nxt, t[q].nxt + 26, t[r].nxt);
            t[q].link = r;
            while (t[p].nxt[c] == q) {
                t[p].nxt[c] = r;
                p = t[p].link;
            }
            return r;
        }
        int cur = ++cntNodes;
        t[cur].len = t[p].len + 1;
        t[cur].siz = 1;
        while (!t[p].nxt[c]) {
            t[p].nxt[c] = cur;
            p = t[p].link;
        }
        t[cur].link = extend(p, c);
        return cur;
    }
};
```

子序列自动机

对于给定的长度为 n 的主串 s，以 $\mathcal{O}(n)$ 的时间复杂度预处理、$\mathcal{O}(m+\log \text{size}{:}s)$ 的复杂度判定长度为 m 的询问串是否是主串的子序列。

自动离散化、自动类型匹配封装

```
template<class T> struct SequenceAutomaton {
    vector<T> alls;
    vector<vector<int>> ver;

    SequenceAutomaton(auto in) {
        for (auto &i : in) {
            alls.push_back(i);
        }
        sort(alls.begin(), alls.end());
        alls.erase(unique(alls.begin(), alls.end()), alls.end());

        ver.resize(alls.size() + 1);
        for (int i = 0; i < in.size(); i++) {
            ver[get(in[i])].push_back(i + 1);
        }
    }
    bool count(T x) {
        return binary_search(alls.begin(), alls.end(), x);
    }
    int get(T x) {
        return lower_bound(alls.begin(), alls.end(), x) - alls.begin();
    }
```

```
    bool contains(auto in) {
        int at = 0;
        for (auto &i : in) {
            if (!count(i)) {
                return false;
            }

            auto j = get(i);
            auto it = lower_bound(ver[j].begin(), ver[j].end(), at + 1);
            if (it == ver[j].end()) {
                return false;
            }
            at = *it;
        }
        return true;
    }
};
```

朴素封装

原时间复杂度中的 $\text{size}:s$ 需要手动设置。类型需要手动设置。

```
struct SequenceAutomaton {
    vector<vector<int>> ver;

    SequenceAutomaton(vector<int> &in, int size) : ver(size + 1) {
        for (int i = 0; i < in.size(); i++) {
            ver[in[i]].push_back(i + 1);
        }
    }
    bool contains(vector<int> &in) {
        int at = 0;
        for (auto &i : in) {
            auto it = lower_bound(ver[i].begin(), ver[i].end(), at + 1);
            if (it == ver[i].end()) {
                return false;
            }
            at = *it;
        }
        return true;
    }
};
```